SANITATION RECORD MANAGEMENT SYSTEM

Technical Blueprint & Comprehensive Architecture (TCA) Report

Prepared by: Bandish Vaidya Project Repository: Sanitation Record Management System
Document Date: 12 October 2026 Technology: Django, Django REST Framework, Python, HTML, CSS, JavaScript, SQLite, PostgreSQL

Document Purpose: To provide a comprehensive technical reference explaining the application's purpose, modules, page navigation, API integration, database architecture, business workflows, security, deployment, and maintenance.

TABLE OF CONTENTS

1. Executive Summary
2. Introduction and Project Objectives
3. Project Scope
4. Technology Stack
5. System Architecture
6. Application Structure
7. User Roles and Permissions
8. Page Inventory and Navigation
9. API Architecture and Integration
10. Database Architecture and Relationships
11. Database Entity Reference
12. Business Logic and Calculations
13. Authentication and Authorization Flow
14. Module-by-Module Technical Explanation
15. File Uploads and PDF Reporting
16. Search and Combined Records
17. Frontend Architecture
18. Testing and Quality Assurance
19. Deployment Architecture
20. Security and Data Protection
21. Known Development Issues and Lessons
22. Troubleshooting Guide

23. Operational Procedures
24. Future Enhancements
25. Glossary
26. Source-Code Verification Checklist
27. Conclusion

1. EXECUTIVE SUMMARY

The Sanitation Record Management System is a web-based application developed to digitize and centralize sanitation-related operations.

The application brings together plant management, sanitizer titration, Chloragel titration, daily sanitation inspections, ATP report uploads, daily combined records, search functionality, PDF reporting, corrective actions, user management, and audit logging.

The system is built using Django as the primary web framework and Django REST Framework (DRF) to provide API functionality. The user interface uses Django templates, HTML, CSS, JavaScript, and Bootstrap-based components. SQLite supports local development, while PostgreSQL is supported for deployment.

The application separates presentation, request processing, business logic, data persistence, and file handling into distinct responsibilities. This structure supports maintainability and makes it easier to add functionality without unnecessarily coupling the different modules.

1.1 Main Capabilities

- Centralized sanitation record management.
- Plant-based organization of operational records.
- Automated sanitizer and Chloragel calculations.
- Daily sanitation inspections.
- ATP report uploads and associated metadata.
- Combined daily records and record searching.
- PDF reporting for supported records.
- Role-based authentication and authorization.
- Corrective action tracking.
- Audit logging for supported activities.
- API-based communication between frontend components and backend services.

1.2 Intended Users

The system supports three primary application roles:

- Admin: Administrative operations and management capabilities.
- Inspector: Sanitation activities and permitted record operations.
- Team Lead: Permitted operational and record-management activities.

The exact permissions for each role are controlled by the application's authentication and permission implementation.

2. INTRODUCTION AND PROJECT OBJECTIVES

2.1 Background

Sanitation operations can produce many records, including chemical titration results, inspection responses, ATP reports, and corrective action information.

When these records are managed manually, several challenges may arise:

- Difficulty finding historical records.
- Inconsistent data entry.
- Repeated calculations.
- Poor visibility into daily activities.
- Difficulty combining records from different sources.
- Time-consuming report preparation.
- Limited traceability of record creation and updates.

The Sanitation Record Management System addresses these challenges by providing a centralized digital application.

2.2 Project Objectives

The primary objectives are:

1. Digitize sanitation recordkeeping.
2. Organize records by plant and date.
3. Standardize supported titration calculations.
4. Simplify daily inspection data entry.
5. Associate ATP reports with relevant operational information.
6. Make historical records searchable.
7. Generate PDF reports for supported record selections.
8. Apply role-based access control.
9. Maintain audit-related information.

10. Provide an architecture that can be maintained and extended.

2.3 Project Repository

GitHub repository:

https://github.com/Bandish04/Sanitation-Record-Management-System

The repository is the primary source of truth for the current implementation, including Python code, templates, static assets, configuration, migrations, tests, and dependency definitions.

3. PROJECT SCOPE

3.1 Included Functionality

The documented project scope includes:

- User authentication.
- User and role management.
- Plant management.
- Sanitizer titrations.
- Chloragel titrations.
- Daily sanitation inspections.
- ATP report uploads.
- Daily combined records.
- Record searching and detail views.
- PDF reporting.
- Corrective actions.
- Audit logs.
- REST API integration.
- Database migrations and deployment configuration.

3.2 Technical Boundaries

The application follows a Django-based architecture rather than requiring a separate React frontend for its documented initial version.

The main technical boundaries are:

1. Browser-based user interface.
2. Django page routing and templates.

3. Django REST Framework APIs.
4. Backend business logic and validation.
5. Django ORM and relational database.
6. Uploaded-file storage.
7. PDF/report generation.
8. Authentication and permissions.

Automatic OCR extraction from ATP documents was not part of the initial implementation plan. Similarly, Docker is not required for the documented local development workflow.

4. TECHNOLOGY STACK

Technology	Purpose
Python	Primary programming language
Django	Web framework and application backend
Django REST Framework	REST API development
JWT authentication	Token-based API authentication where configured
HTML5	Page structure
CSS3	Styling and layout
JavaScript	Browser interactions and API requests
Bootstrap	Responsive interface components

TailAdmin-style CSS	Application styling
SQLite	Local development database
PostgreSQL	Deployment database support
Django ORM	Database access and object relationships
PDF-generation libraries	Generating downloadable reports
Git	Version control
GitHub	Source-code hosting
Django Test Framework	Automated testing

The exact installed package versions should be taken from the current `requirements.txt` file.

The development environment previously recorded for this project used Python 3.14.7 and Django 6.1.1.

5. SYSTEM ARCHITECTURE

5.1 Architectural Overview

The system follows a layered web application architecture.

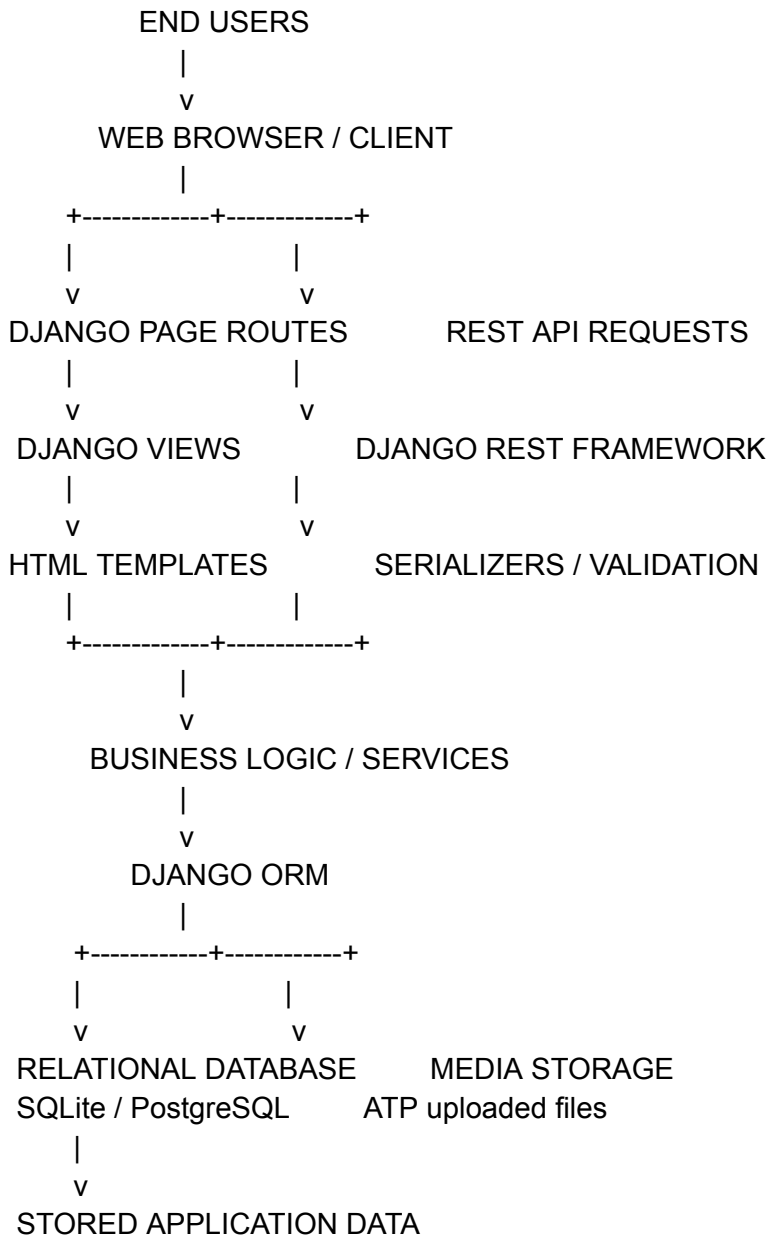
A user interacts with the browser-based interface. The browser loads Django-rendered pages and may use JavaScript to call backend API endpoints.

Django routes requests to the appropriate page views or API views. The backend validates input, applies permissions, executes business rules, and uses the Django ORM to retrieve or save data.

The database stores structured records. Uploaded ATP files are handled by media storage, while PDF-generation functionality produces downloadable report files.

5.2 High-Level Architecture Diagram

Copy this diagram into Word using a monospaced font such as Consolas if you want to preserve its alignment.



REPORTING
|
v
PDF GENERATION
|
v
DOWNLOADABLE PDF FILE

5.3 Responsibilities of Each Layer

A. Presentation Layer

Responsible for:

- Rendering pages.
- Displaying forms and tables.
- Providing navigation.
- Showing record details.
- Handling modals and other interactions.
- Displaying success and error messages.

Main technologies: Django templates, HTML, CSS, JavaScript, and Bootstrap.

B. Routing Layer

Responsible for matching incoming URLs to the appropriate page view or API endpoint.

Relevant source files typically include:

- `config/urls.py`
- Application-specific `urls.py` files.

C. API Layer

Responsible for:

- Receiving HTTP requests.
- Parsing request data.
- Calling serializers and business logic.
- Enforcing authentication and permissions.
- Returning structured responses or files.

D. Business Logic Layer

Responsible for domain operations, such as:

- Titration calculations.
- Draft and submission handling.
- Record filtering.
- Result-status evaluation.
- Report preparation.
- Validation of operational rules.

E. Data Access Layer

The Django ORM maps Python model objects to relational database records.

It handles database queries, record creation, updates, and relationships.

F. Storage and Reporting Layer

Responsible for:

- Persisting uploaded files.
- Retrieving files when authorized.
- Preparing PDF reports.
- Returning generated files to the user.

5.4 Architectural Design Principle

The frontend should not be treated as the authority for security or business rules.

For example, hiding a Delete button from an Inspector improves the interface, but it does not prevent that user from directly calling a delete API. The backend must also reject unauthorized requests.

6. APPLICATION STRUCTURE

6.1 Main Directory Structure

The project has been organized into Django applications, shared templates, and static assets.

```
Sanitation-Record-Management-System/
│
├── accounts/
│   ├── models.py
```

```
│   ├── views.py
│   ├── serializers.py
│   ├── permissions.py
│   └── urls.py
│
├── plants/
│   ├── models.py
│   ├── views.py
│   ├── serializers.py
│   └── urls.py
│
├── titrations/
│   ├── models.py
│   ├── views.py
│   ├── serializers.py
│   └── urls.py
│
├── inspections/
│   ├── models.py
│   ├── views.py
│   ├── serializers.py
│   └── urls.py
│
├── records/
│   ├── models.py
│   ├── views.py
│   ├── serializers.py
│   └── urls.py
│
├── corrective_actions/
│
├── audit/
│
├── config/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
│
├── templates/
│   ├── base.html
│   ├── dashboard.html
│   ├── plants/
│   ├── titrations/
│   ├── inspections/
```

```
│   ├── records/
│   ├── corrective_actions/
│   ├── audit/
│   ├── admin/
│   └── registration/
│
├── static/
│   └── css/
│       └── tailadmin.css
│
├── media/
├── manage.py
├── requirements.txt
├── .gitignore
└── README.md
```

Note: This is a representative structure based on the project's documented organization. Confirm the current files and exact application boundaries in the repository.

6.2 Important Project Files

File or directory	Responsibility
`manage.py`	Runs Django management commands
`config/settings.py`	Application settings, database, authentication, static files, and installed apps
`config/urls.py`	Main URL routing
`config/wsgi.py`	WSGI entry point used by production application servers
`requirements.txt`	Python package dependencies

`templates/`	HTML templates
`static/`	CSS, JavaScript, and other static assets
`media/`	Uploaded files during local development
`models.py`	Database model definitions
`serializers.py`	API data validation and representation
`views.py`	Page handling and API request processing
`permissions.py`	Custom authorization rules
`migrations/`	Versioned database schema changes

7. USER ROLES AND PERMISSIONS

The application uses role-based access control through Django authentication and groups.

The three documented roles are Admin, Inspector, and Team Lead.

7.1 Admin

The Admin role is intended to provide broad application management capabilities.

Typical responsibilities include:

- Managing plants.

- Managing users.
- Accessing operational modules.
- Viewing and managing permitted records.
- Reviewing audit information.
- Accessing reporting functionality.

Important distinction: Membership in an `Admin` group is not automatically equivalent to having Django's `is_superuser=True` flag. These are separate mechanisms and must be configured appropriately.

7.2 Inspector

The Inspector role supports sanitation operations.

Typical activities include:

- Entering permitted titration measurements.
- Performing daily inspections.
- Creating or submitting records where authorized.
- Viewing operational records permitted by the application.

7.3 Team Lead

The Team Lead role supports operational oversight and permitted record-management activities.

Depending on the backend rules, these activities may include:

- Creating or editing permitted titration records.
- Submitting records.
- Reviewing sanitation information.
- Accessing relevant operational reports.

7.4 Role and Permission Matrix

The following matrix describes intended access and must be checked against the current backend implementation.

Function	Admin	Inspector	Team Lead

Login	Yes	Yes	Yes
View dashboard	Yes	Yes	Yes
View plants	Yes	Yes	Yes, if permitted
Manage plants	Administrative access	Read-only intended	Depends on configured permissions
Create titrations	Depends on endpoint permission	Depends on endpoint permission	Depends on endpoint permission
Edit titrations	Depends on endpoint permission	Depends on endpoint permission	Depends on endpoint permission
Submit titrations	Depends on endpoint permission	Depends on endpoint permission	Depends on endpoint permission
Perform inspections	As permitted	As permitted	As permitted
Upload ATP reports	As permitted	As permitted	As permitted
View combined records	As permitted	As permitted	As permitted
Manage users	Administrative access	Restricted	Restricted

View audit logs	As permitted	Restricted	Restricted

7.5 Custom Permission Classes

The project includes custom permission classes such as:

- `IsInspector`
- `IsTeamLead`
- `IsAdmin`

These classes can restrict access to API operations based on the authenticated user's group membership.

A permission class must be configured on the appropriate endpoint. Merely defining a permission class does not automatically apply it to every API.

7.6 Security Recommendation

The UI and backend must agree on access rules.

If the interface displays a button that the API rejects, users encounter errors. If the interface hides a button but the API allows unauthorized access, the application has a security weakness.

Both layers should be tested independently.

8. PAGE INVENTORY AND NAVIGATION

This section explains the primary pages and how a user moves between them.

The routes below are recorded examples from the project context. The exact URL patterns should be confirmed in the current URL configuration.

8.1 Login Page

Template: `templates/registration/login.html`

Purpose:

- Accept user credentials.

- Initiate authentication.
- Redirect or navigate users to the appropriate page.
- Prevent unauthenticated access to protected operations.

The exact login endpoint and token behavior depend on the configured authentication implementation.

8.2 Dashboard

Template: `templates/dashboard.html`

Purpose:

The dashboard acts as the main navigation point for the application.

It provides access to the principal modules:

- Plants Management.
- Sanitizer Titrations.
- Chloragel Titrations.
- Inspections.
- ATP Reports.
- Daily Records.
- Search Records.
- Corrective Actions.
- User Management.
- Audit Logs.

8.3 Plants Management

Template: `templates/plants/list.html`

Purpose:

- Display available plants.
- Show plant information and status.
- Provide management controls to authorized users.
- Supply plant context for sanitation records.

The plant relationship is important because operational records are commonly organized by the plant where the activity occurred.

8.4 Sanitizer Titrations

Template: `templates/titrations/sanitizer.html`

Purpose:

- Display sanitizer titration records.
- Capture drop counts.
- Calculate the derived measurements.
- Save drafts.
- Submit records.
- Display statuses and relevant record information.

8.5 Chloragel Titrations

Template: `templates/titrations/chloragel.html`

Purpose:

- Display Chloragel titration records.
- Capture measurements.
- Calculate the result percentage.
- Save and submit records.
- Display status information.

8.6 Daily Sanitation Inspections

Template: `templates/inspections/inspection.html`

Purpose:

- Display inspection questions.
- Capture inspection responses.
- Associate the inspection with the relevant operational context.
- Save inspection information.

The actual questions, answer options, and submission rules are defined by the implemented inspection models and frontend/backend logic.

8.7 ATP Reports

Template: `templates/inspections/atp.html`

Purpose:

- Upload ATP report files.
- Associate reports with plants.
- Optionally associate a report with an inspection.
- Store notes and uploader information.
- Display existing report records.

The documented upload interface supports formats including PDF, JPG, JPEG, PNG, and WebP, with a reported maximum file size of 10 MB. Confirm the current validation rules before treating these as production guarantees.

8.8 Daily Combined Records

Template: `templates/records/daily.html`

Purpose:

Display records from several modules for a selected plant and date.

The combined view may include:

- Sanitizer titrations.
- Chloragel titrations.
- Inspections.
- ATP reports.

The combined page does not necessarily mean these records are stored in one database table. They can remain separate entities and be combined when queried for display.

8.9 Search Records

Template: `templates/records/search.html`

Purpose:

- Search available records.
- Filter by plant.
- Filter by date range.
- Filter by record type and status where supported.
- Open detailed record views.

Because the application contains multiple record types, the detail interface must interpret the correct fields for each type.

8.10 PDF Reports

Template: `templates/records/pdf_reports.html`

Purpose:

- Select supported report criteria.
- Request report generation.
- Download or view the resulting PDF.

The PDF page may use a separate API endpoint to generate the actual file.

8.11 Corrective Actions

Template: `templates/corrective_actions/list.html`

Purpose:

Support the management of corrective actions associated with sanitation operations.

The precise relationship between corrective actions and source records must be verified in the models.

8.12 User Management

Template: `templates/admin/users.html`

Purpose:

- Display user information.
- Support authorized account-management operations.
- Associate users with roles or groups where implemented.

8.13 Audit Logs

Template: `templates/audit/logs.html`

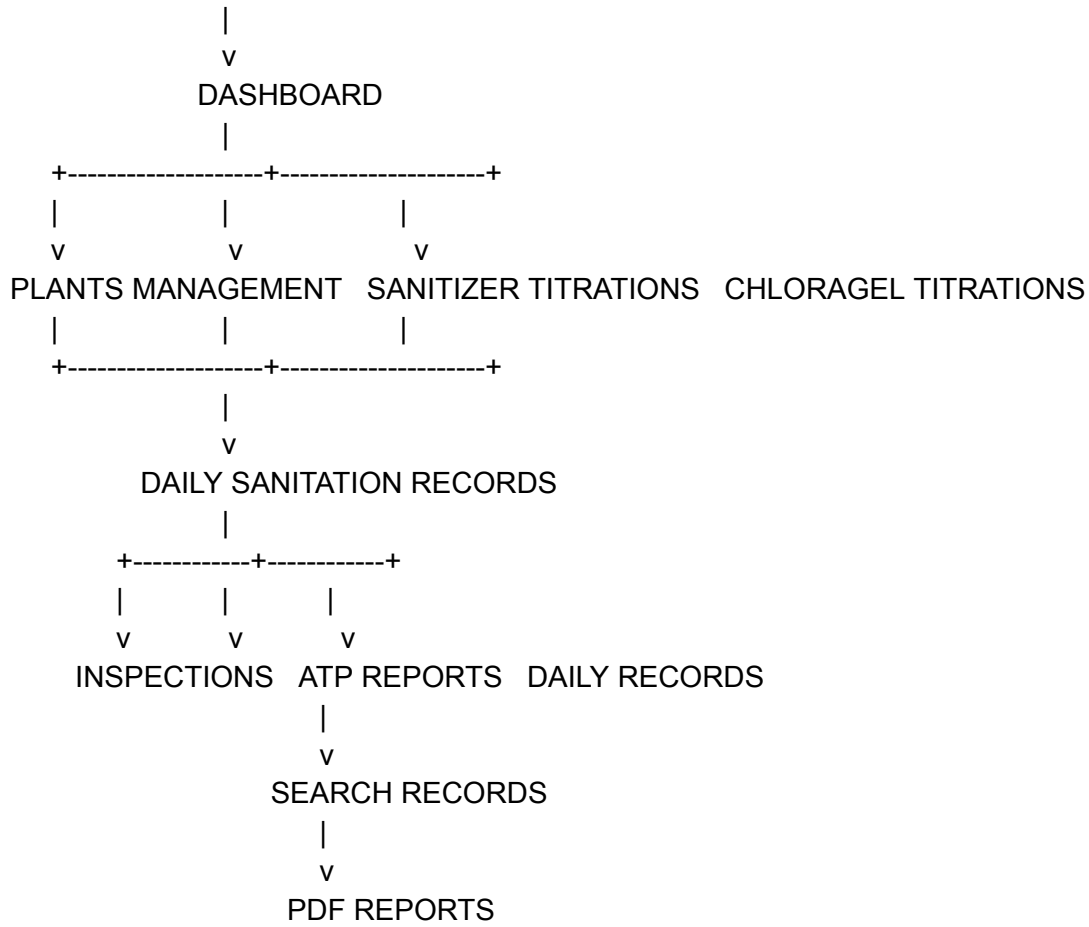
Purpose:

Display supported audit events to authorized users.

Audit information helps with operational traceability and troubleshooting.

8.14 Navigation Flow Diagram

LOGIN



The diagram represents the logical navigation and data workflow, not an assertion that every page redirects directly to the next page.

9. API ARCHITECTURE AND INTEGRATION

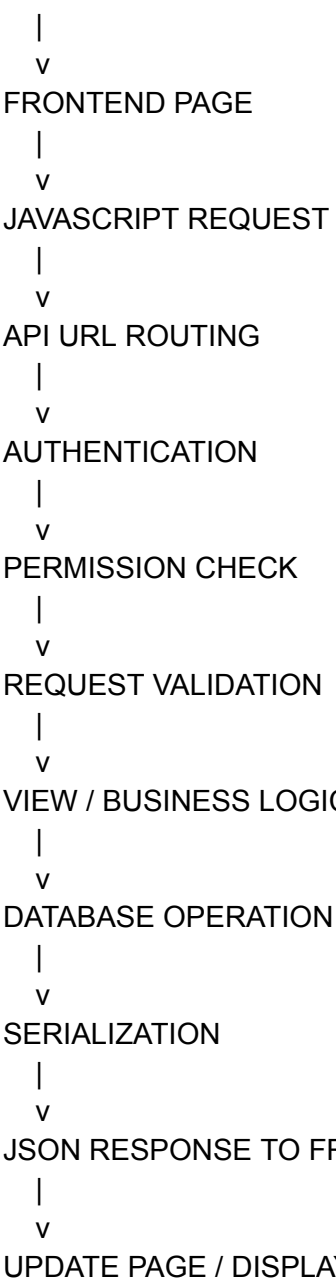
9.1 What Is an API?

An Application Programming Interface (API) defines how one part of an application communicates with another.

In this project, Django REST Framework provides API functionality for supported application operations.

The frontend can send HTTP requests to the backend and receive structured responses. Depending on the operation, these responses may contain JSON data, validation errors, status information, or downloadable files.

9.2 API Request Flow



For a PDF request, the response may instead contain a PDF file rather than JSON.

9.3 API Responsibilities

The API layer can handle:

- Receiving data from forms.
- Retrieving records.
- Validating input.
- Creating and updating records.
- Applying permissions.
- Calculating or returning derived values.
- Uploading files.
- Generating reports.
- Returning errors when requests fail.

9.4 Representative API Endpoints

These are documented endpoint examples from prior project work. Verify each route against the current source code.

HTTP method	Endpoint	Purpose
POST	`/api/auth/login/`	Authenticate a user, if configured at this path
GET	`/api/auth/users/`	Retrieve user information, subject to permissions
GET	`/api/records/daily/`	Retrieve combined daily records
GET	`/api/records/daily/pdf/`	Generate or retrieve a PDF report

Additional endpoints may exist for plants, titrations, inspections, ATP reports, corrective actions, and audit logs.

9.5 Daily Records API Example

Example request:

GET /api/records/daily/?plant=1&date=2026-10-12

The query parameters identify:

- `plant=1`: the selected plant's identifier.
- `date=2026-10-12`: the requested record date.

The exact response fields depend on the serializer and view implementation.

9.6 PDF API Example

GET /api/records/daily/pdf/?plant=1&date=2026-10-12

This illustrates a request for a report using the same plant/date selection.

The endpoint should validate the requested data and return the appropriate report response.

9.7 HTTP Status Codes

Status	Meaning	Typical cause
200	OK	Request completed successfully
201	Created	Record created successfully
400	Bad Request	Invalid input or failed validation
401	Unauthorized	Authentication missing or invalid
403	Forbidden	User lacks required permission

404	Not Found	Requested route or record does not exist
500	Internal Server Error	Unexpected backend exception

The exact status codes depend on the API implementation.

9.8 API Testing

Recommended tools include:

- Postman for manually testing endpoints.
- Browser Developer Tools for inspecting requests and responses.
- Django's development-server output for backend error traces.
- Django automated tests for repeatable verification.

When a page fails to save a record, inspect the request payload, response status, response body, and server logs before assuming the database is responsible.

10. DATABASE ARCHITECTURE AND RELATIONSHIPS

This is one of the most important sections of the technical blueprint.

It explains how application records are conceptually connected and how to interpret the database relationships.

Important: The diagram below is a logical model based on the project's documented modules. It is not a verified entity-relationship diagram extracted from the current database migrations.

10.1 Database Technology

The project supports two database environments.

SQLite

Used for local development.

Advantages include:

- Minimal setup.
- No separate database server required.
- Convenient development and testing.

PostgreSQL

Supported for deployment.

Advantages include:

- A dedicated database server.
- Strong relational database capabilities.
- Better suitability for many production workloads.
- Independent persistence from the application process.

Moving from SQLite to PostgreSQL does not automatically transfer records. Database schema creation and data migration are separate operations.

10.2 How Django Connects to the Database

Django uses its Object-Relational Mapper (ORM).

Instead of writing every SQL query manually, application code can use Python model objects and querysets.

For example, a simplified model query might look like this:

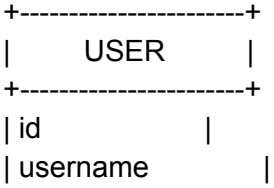
Plant.objects.all()

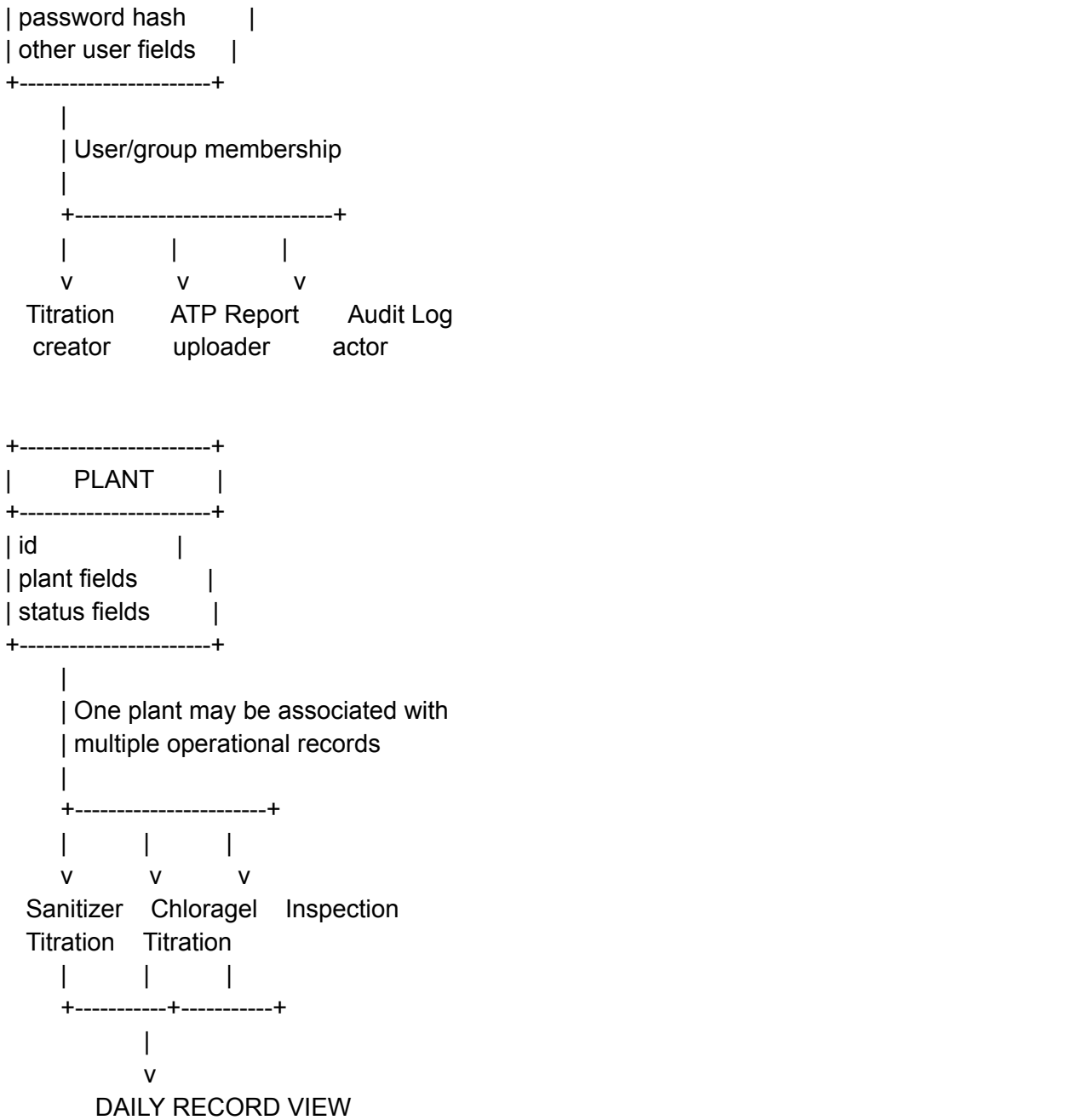
A record might be created using:

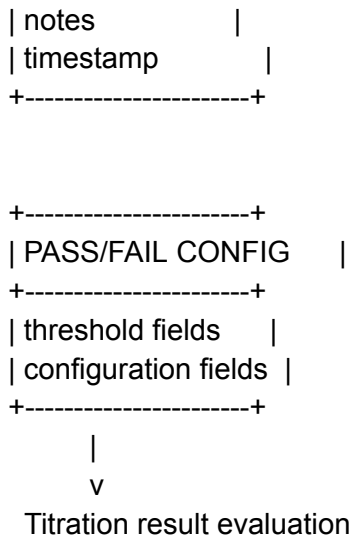
SanitizerTitration.objects.create(plant=plant, drops=4)

These examples illustrate the ORM concept only. The actual model names and field names must be confirmed in the project source code.

10.3 Logical Entity-Relationship Diagram




| password hash |
| other user fields |
+----------+
|
| User/group membership
|
+----------+
| | |
v v v
Titration ATP Report Audit Log
creator uploader actor
+----------+
| PLANT |
+----------+
| id |
| plant fields |
| status fields |
+----------+
|
| One plant may be associated with
| multiple operational records
|
+----------+
| | |
v v v
Sanitizer Chloragel Inspection
Titration Titration
| | |
+----------+----------+
|
v
DAILY RECORD VIEW
+----------+
| ATP REPORT |
+----------+
| id |
| plant reference |
| optional inspection |
| uploader reference |
| file reference |



The relationships and attributes in this diagram are conceptual. Confirm exact model definitions and fields before using this diagram as a formal database specification.

10.4 Primary Keys

A primary key uniquely identifies a record in a database table.

For example, a plant may have a database identifier such as 1, while another plant may have identifier 2.

These identifiers are used internally to reference records.

A user interface should generally show the plant's name rather than displaying only its numeric ID.

10.5 Foreign Keys

A foreign key links one database record to another.

Consider the following conceptual example:

PLANT TABLE

id	plant_name
1	Plant A
2	Plant B

SANITIZER TITRATION TABLE

id	plant_id	drops
10	1	4
11	2	8

In this example:

- Titration `10` belongs to Plant A.
- Titration `11` belongs to Plant B.
- The `plant_id` field identifies the related plant.

The exact table and column names may differ in the implementation.

10.6 User Relationships

Operational records commonly need to identify the person who created or uploaded them.

Conceptually:

USER TABLE

id	username
1	Admin
2	Ajay
3	Deep

TITRATION TABLE

id	created_by_id
10	2
11	3

This lets the application associate a record with its creator.

The frontend can resolve the relationship and display `Ajay` or `Deep` rather than displaying only `2` or `3`.

10.7 Plant Relationships

Plant information provides a common operational context for many records.

The expected logical relationships are:

- A plant can be associated with multiple sanitizer titrations.
- A plant can be associated with multiple Chloragel titrations.
- A plant can be associated with multiple inspections.
- A plant can be associated with multiple ATP reports.
- A plant may be associated with corrective actions.

The exact number of records allowed and the actual foreign-key constraints depend on the model definitions.

10.8 Inspection and ATP Relationships

An ATP report can be associated with a plant and may optionally be associated with an inspection.

Conceptually:

```
PLANT
  |
  +---- ATP REPORT A
  |
  +---- ATP REPORT B
  |
  +---- INSPECTION A
          |
          +---- ATP REPORT C
```

This is a logical illustration, not proof that every ATP report must be linked to an inspection.

10.9 Pass/Fail Configuration

The project includes a `PassFailConfiguration` model in its documented implementation context.

This configuration is intended to provide thresholds used when evaluating titration results.

The exact fields, matching rules, and behavior when multiple configurations exist must be confirmed in the source code.

10.10 Audit Relationships

An audit log may associate an event with:

- The user who performed the action.
- The action type.
- A related object or record.
- A description.
- A timestamp.

The precise structure depends on the `audit` application's model and logging implementation.

10.11 Why the Combined Daily Records Page Does Not Require One Giant Table

The daily records interface can combine records from different tables without storing every record in a single database table.

For example:

```
Sanitizer Titrations ----+
                         |
Chloragel Titrations ----+
                         |
Inspections -------------+----> DAILY RECORDS VIEW
                         |
ATP Reports -------------+
```

The backend can query each source and return a combined response.

This design preserves the individual structure of each record type while allowing the frontend to present a unified daily overview.

10.12 Database Migrations

Django migrations track changes to the database schema.

Typical commands are:

```
python manage.py makemigrations
python manage.py migrate
```

`makemigrations` generates migration files from model changes. `migrate` applies migrations to the configured database.

Migrations create or modify tables and constraints. They do not automatically copy local records into a different database.

11. DATABASE ENTITY REFERENCE

This section provides a conceptual reference for the principal data entities.

Entity	Purpose	Important information to verify
User	Stores user identity and authentication information	Username, password hash, account status
Group	Defines role membership	Admin, Inspector, Team Lead
Plant	Represents an operational plant	Name, status, other plant details
Sanitizer Titration	Stores sanitizer measurement records	Drop count, calculated values, status, creator
Chloragel Titration	Stores Chloragel measurements	Drop count, calculated percentage, status, creator
Inspection	Represents an inspection event	Plant, inspector, date, responses
Inspection Template	Defines inspection structure, if implemented as a separate model	Template name and related questions

ATP Report	Stores ATP report metadata and file reference	Plant, optional inspection, uploader, notes, file
PassFailConfiguration	Provides result thresholds	Minimum and maximum values, matching criteria
Corrective Action	Tracks follow-up activities	Status, owner, dates, related source record
Audit Log	Records supported events	Actor, action, target record, timestamp

Database documentation requirement: Before using this table for database administration or formal sign-off, verify the actual fields, constraints, relationships, and model names in the current source code.

12. BUSINESS LOGIC AND CALCULATIONS

12.1 Sanitizer / Maxquat Titration

The documented calculation rules are:

Parameter	Value
Sample volume	5 mL
Supported drop-count range	1–32 drops

PPM formula	Drops × 12.5
Percentage formula	Drops × 0.0125

Example

Given four drops:

PPM = 4 × 12.5

PPM = 50

Percentage = 4 × 0.0125

Percentage = 0.05%

Conceptual Workflow

Select plant
  |
  v
Enter drop count
  |
  v
Validate permitted range
  |
  v
Calculate PPM and percentage
  |
  v
Evaluate configured thresholds
  |
  v
Save draft or submit
  |
  v
Display stored result

The backend should be responsible for enforcing validation and authoritative business rules.

12.2 Chloragel Titration

The documented calculation rules are:

Parameter	Value
Sample volume	15 mL
Supported drop-count range	1–50 drops
Result formula	Drops × 0.198

Example

Given ten drops:

Result percentage = 10 × 0.198

Result percentage = 1.98%

Conceptual Workflow

Select plant
  |
  v
Enter drop count
  |
  v
Validate permitted range
  |
  v
Calculate result percentage
  |
  v
Evaluate configured thresholds
  |
  v
Save draft or submit
  |
  v

Display stored result

12.3 Result Statuses

The recorded application context includes these statuses:

- `PASS`
- `FAIL`
- `REVIEW_REQUIRED`

The intended meaning is:

Status	Meaning
PASS	Result satisfies the applicable configured criteria
FAIL	Result does not satisfy the applicable configured criteria
REVIEW_REQUIRED	A definitive result cannot be assigned under the configured rules

The exact evaluation logic must be confirmed in the implementation.

If required configuration is unavailable, the application may use `REVIEW_REQUIRED` rather than assigning an unsupported result.

12.4 Draft and Submission Workflow

A draft is an incomplete or unsubmitted record.

Submission represents a change in the record's workflow state.

The intended lifecycle is:

```
CREATE RECORD
    |
    v
```

SAVE DRAFT
    |
    v
EDIT IF PERMITTED
    |
    v
SUBMIT RECORD
    |
    v
LOCK ORDINARY EDITING

The backend must enforce submission rules. Disabling an Edit button in the browser alone does not prevent unauthorized updates through an API.

12.5 Date and Time Handling

Daily records depend on correct date filtering.

For example, a record created near midnight may have different calendar dates depending on its stored timezone and the timezone used to display or filter it.

The application should consistently handle:

- Database timestamp storage.
- Django timezone settings.
- Local display time.
- Date filters.
- Day-boundary calculations.
- PDF report date selection.

A record appearing under the wrong day should trigger an investigation of timezone conversion and query boundaries before changing the UI.

13. AUTHENTICATION AND AUTHORIZATION FLOW

13.1 Authentication

Authentication verifies the user's identity.

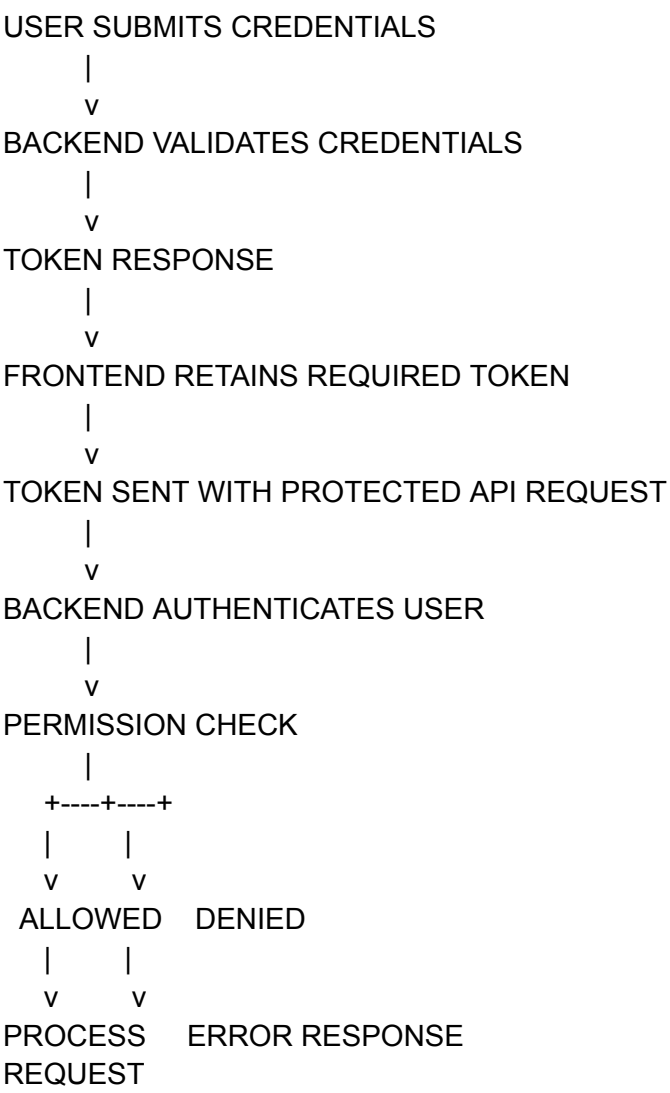
A typical flow is:

1. The user opens the login page.
2. The user submits credentials.
3. Django or the configured authentication API validates them.
4. The application establishes the configured authentication state.
5. The user accesses permitted pages and endpoints.

13.2 JWT Authentication

The project context records JWT-based API authentication.

A typical JWT flow is:



The precise token format, header, expiration, refresh mechanism, and storage method depend on the current implementation.

13.3 Authentication Versus Authorization

These terms describe different operations.

Authentication: Who is making the request?

Authorization: Is that user allowed to perform the requested operation?

A user can authenticate successfully and still receive a `403 Forbidden` response if their role does not have the required permission.

13.4 Example

Suppose an authenticated user attempts to create a titration.

1. The request reaches the titration API.
2. The backend identifies the user.
3. The endpoint checks the user's permissions.
4. The backend accepts or rejects the operation.
5. If accepted, the record is validated and processed.

A visible button does not guarantee that the request will succeed. The frontend and backend permission rules must be consistent.

14. MODULE-BY-MODULE TECHNICAL EXPLANATION

14.1 Accounts Module

Responsibilities include:

- Authentication integration.
- User management.
- Group membership.
- Role-based permission support.
- User-related API operations.

Important files to inspect:

- `accounts/models.py`
- `accounts/views.py`

- `accounts/serializers.py`
- `accounts/permissions.py`
- `accounts/urls.py`

14.2 Plants Module

Responsibilities include:

- Storing plant information.
- Listing plants.
- Supporting permitted management actions.
- Supplying plant references for operational records.

The plant model is an important domain entity because it provides context for titrations, inspections, and ATP reports.

14.3 Titrations Module

Responsibilities include:

- Sanitizer titration operations.
- Chloragel titration operations.
- Calculation logic.
- Draft and submission workflows.
- Result-status evaluation.
- Record attribution and timestamps.

The exact model separation and API structure should be confirmed in the `titrations` application.

14.4 Inspections Module

Responsibilities include:

- Inspection record creation.
- Inspection response handling.
- Inspection retrieval.
- ATP report functionality where configured within this application.

The interface and backend should use consistent question labels and response values.

14.5 Records Module

Responsibilities include:

- Combining records from multiple source modules.
- Filtering by plant and date.
- Searching records.
- Displaying details.
- Generating PDF reports.

This module provides a consolidated view without necessarily replacing the underlying source records.

14.6 Corrective Actions Module

Responsibilities include:

- Managing corrective action information.
- Tracking action status.
- Supporting follow-up activities.

The exact relationship between a corrective action and its originating inspection or other record must be verified in the model definitions.

14.7 Audit Module

Responsibilities include:

- Storing supported audit events.
- Recording relevant action descriptions.
- Associating events with actors and timestamps where implemented.
- Displaying audit information to authorized users.

Audit logs are useful for tracing application activity, but the scope of logging depends on which operations are instrumented.

15. FILE UPLOADS AND PDF REPORTING

15.1 ATP File Uploads

An ATP report usually involves two separate elements:

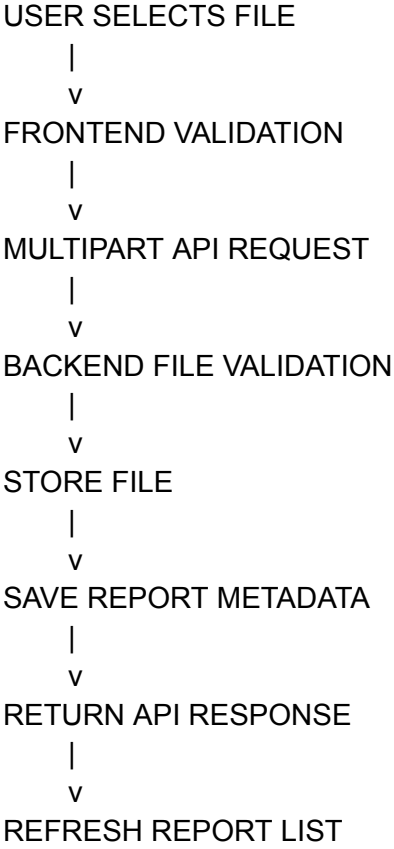
1. A database record containing report metadata.
2. A file stored in media storage.

The metadata may contain:

- Plant reference.
- Optional inspection reference.
- Uploader.
- Notes.
- Creation timestamp.
- File reference or path.

The exact fields must be checked against the current model.

15.2 Upload Processing Flow



Frontend validation helps prevent accidental mistakes. Backend validation remains essential because requests can be sent without using the browser interface.

15.3 Media Storage Versus Static Files

These terms should not be confused.

Static files are application assets, such as:

- CSS files.
- JavaScript files.
- Images shipped with the application.

Media files are uploaded by users, such as ATP report documents.

In production, uploaded files may require persistent disk storage or object storage. They should not be assumed to survive an application redeployment unless the storage system guarantees persistence.

15.4 PDF Reporting

The reporting workflow generally involves:

1. Receiving report criteria.
2. Retrieving the matching records.
3. Formatting the data.
4. Generating a PDF.
5. Returning the file to the requester.

The generated report should be checked for:

- Correct plant selection.
- Correct date range.
- Correct record types.
- Correct user and plant names.
- Consistent calculations.
- Appropriate handling of missing optional values.
- Readable tables and page layouts.

16. SEARCH AND COMBINED RECORDS

16.1 Search Functionality

The Search Records page provides a centralized way to locate information across supported record types.

Filters may include:

- Plant.
- Start date.
- End date.

- Record type.
- Status.

The exact filters depend on the current implementation.

16.2 Combined Daily Records

The daily records endpoint is documented as returning information from several sources, including:

- `sanitizer_titrations`
- `chloragel_titrations`
- `inspections`
- `atp_reports`

A combined API response can conceptually be represented as:

```
{
  "sanitizer_titrations": [],
  "chloragel_titrations": [],
  "inspections": [],
  "atp_reports": []
}
```

This is an illustrative structure, not a guaranteed response schema.

16.3 How the Frontend Displays Combined Data

The frontend receives the response and renders the relevant records.

For example:

```
DAILY RECORDS PAGE
        |
        v
REQUEST RECORDS FOR PLANT + DATE
        |
        v
API RETURNS MULTIPLE RECORD TYPES
        |
        +------> Sanitizer records
        |
        +------> Chloragel records
        |
```

```
        +------> Inspection records
        |
        +------> ATP report records
        |
        v
FRONTEND DISPLAYS COMBINED VIEW
```

16.4 Handling Missing Fields

Different record types have different fields.

For example, an ATP report may contain a file reference and notes, while a titration contains a drop count and calculated result.

A single detail renderer must therefore interpret the correct fields for each source type.

If a detail popup displays − despite a record containing information, possible causes include:

- The API returns a differently named field.
- The frontend reads the wrong property.
- A related object is nested in another object.
- The source record is not included in the response.
- A value is genuinely absent.
- The detail renderer selects the wrong record type.

The correct debugging sequence is to inspect the actual API response and then compare it with the JavaScript or template mapping.

17. FRONTEND ARCHITECTURE

17.1 Template-Based Rendering

The frontend uses Django templates rather than requiring a separate React application.

The shared `base.html` template provides the common layout. Other templates render module-specific content.

17.2 Shared Layout

The shared template typically provides:

- Navigation.

- Page structure.
- Common CSS and JavaScript assets.
- Shared visual components.
- Layout consistency.

17.3 CSS and JavaScript

The documented styling file is:

`static/css/tailadmin.css`

CSS controls the visual presentation, while JavaScript handles interactions such as:

- Opening and closing forms.
- Handling modals.
- Sending API requests.
- Updating displayed records.
- Showing validation messages.
- Managing loading and success states.

The exact implementation depends on the current template files.

17.4 Frontend and Backend Responsibilities

Responsibility	Primary layer
Page layout	Templates and CSS
Button and modal interactions	JavaScript and templates
Displaying API responses	JavaScript and templates
Authentication	Django authentication and configured API authentication
Permission enforcement	Backend permission classes

Data validation	Backend serializers and business logic
Database writes	Backend ORM operations
Titration calculations	Backend business logic where implemented
PDF generation	Backend reporting logic

17.5 Current Maintenance Principle

The established maintenance preference for this project is to preserve backend logic and focus on templates and CSS when the problem is purely visual.

If a defect is caused by backend permissions, data processing, or API behavior, it should be identified separately rather than concealed by a template change.

18. TESTING AND QUALITY ASSURANCE

18.1 Purpose of Testing

Testing helps ensure that changes do not break existing functionality.

The application should be tested at several levels:

- Model and business-rule testing.
- API testing.
- Authentication and permission testing.
- Frontend interaction testing.
- Integration testing.
- Deployment verification.

18.2 Useful Django Commands

Run Django's configuration check:

python manage.py check

Run the complete test suite:

python manage.py test

Run tests for an individual application:

python manage.py test accounts

The last command is an example; replace `accounts` with another application name as needed.

18.3 Test Matrix

Area	Recommended tests
Login	Valid and invalid credentials
Authentication	Missing, invalid, or expired tokens
Permissions	Admin, Inspector, and Team Lead access
Plants	Listing and authorized management operations
Sanitizer	Formula accuracy and drop-count validation
Chloragel	Formula accuracy and drop-count validation
Drafts	Saving and retrieving drafts
Submission	Status transition and edit restrictions

Inspections	Answer validation and record association
ATP reports	File type, size, metadata, and file access
Daily records	Plant/date filtering and source-record inclusion
Search	Filters and detail rendering
PDF	Correct records, readable output, and selected dates
Audit	Actor, event, and timestamp where supported
Deployment	Database connection, static files, media, and environment settings

18.4 Historical Test Information

During earlier development, the project was reported to have a full test suite with 37 passing tests at one point.

This is historical information, not a claim that the current repository necessarily has 37 passing tests. Run the current test suite to establish its present status.

18.5 Regression Testing

After a change, test the affected functionality and nearby workflows.

For example, after changing the Search Records template, verify:

1. Search filters still work.
2. Results still load.
3. Each record type displays its correct details.
4. Links and buttons still work.

5. Permission restrictions remain intact.
6. The browser console shows no unexpected errors.

19. DEPLOYMENT ARCHITECTURE

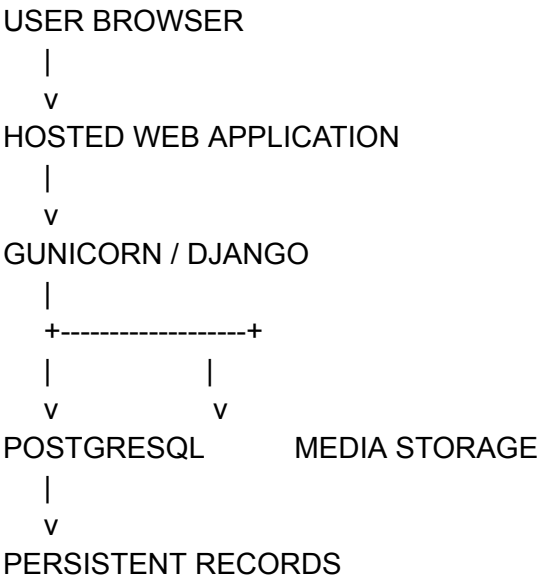
19.1 Deployment Components

The project has previously been configured for deployment using Render and PostgreSQL, with Gunicorn and static-file handling configuration.

The deployment environment should be verified before using these details as a current operational guide.

Component	Purpose
GitHub	Stores the source code
Web hosting platform	Runs the Django application
Gunicorn	WSGI application server
PostgreSQL	Persistent relational database
Environment variables	Supply secrets and environment-specific configuration
Static-file configuration	Serves CSS, JavaScript, and other static assets
Media storage	Stores uploaded reports persistently

19.2 Deployment Request Flow



This is a logical overview; the actual network topology depends on the hosting provider's configuration.

19.3 Environment Variables

The deployment configuration has used variables such as:

Variable	Purpose
`DATABASE_URL`	Database connection
`DJANGO_SECRET_KEY`	Django cryptographic secret
`DJANGO_DEBUG`	Controls debug mode
Deployment administrator variables, if configured	Support a custom administrator setup command

The exact variable names and their behavior must be confirmed in `config/settings.py` and any custom management commands.

19.4 Typical Deployment Commands

The following commands illustrate common deployment operations:

```
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py collectstatic --noinput
gunicorn config.wsgi:application
```

The actual hosting provider's build and start commands may differ.

Do not run a user-creation command automatically during every application startup without confirming its behavior and idempotency.

19.5 Deployment Checklist

Before deployment:

- Confirm environment variables.
- Disable debug mode.
- Configure allowed hosts and CSRF origins.
- Verify the production database connection.
- Apply migrations.
- Configure static-file handling.
- Configure persistent media storage.
- Verify authentication and role permissions.
- Test PDF generation.
- Test file uploads and downloads.
- Review application logs.
- Ensure secrets are not exposed.

19.6 Database Migration Considerations

A newly created PostgreSQL database starts independently of the local SQLite database.

Applying migrations creates the required schema, but does not automatically transfer:

- Local users.
- Plants.
- Titrations.

- Inspections.
- ATP reports.
- Other development records.

If data migration is required, use an intentional and tested data-transfer procedure.

20. SECURITY AND DATA PROTECTION

20.1 Credentials

Passwords, database URLs, Django secret keys, and authentication tokens must not be published in the repository.

The public README should identify demo usernames and explain how to create local accounts, without including actual passwords.

Credentials that have been exposed should be rotated if they are used in a live environment.

20.2 Password Storage

Django's standard authentication system stores password hashes rather than plaintext passwords.

Never add custom code that stores passwords directly in plaintext.

20.3 Role-Based Access

Every protected operation should verify authorization on the backend.

Examples include:

- User management.
- Plant deletion.
- Titration submission.
- Inspection modification.
- ATP report access.
- Audit log viewing.

20.4 File Upload Security

File uploads should be validated for:

- Permitted file types.
- Maximum file size.
- File content and storage handling.
- Appropriate access permissions.
- Safe handling of file names and paths.

File extensions alone are not a complete security check.

20.5 Production Security

Recommended controls include:

- HTTPS.
- `DEBUG=False`.
- Strong secret keys.
- Restricted database credentials.
- Secure cookie configuration.
- Appropriate host and CSRF configuration.
- Protected uploaded files.
- Regular backups.
- Dependency updates.
- Safe error responses.
- Limited logging of sensitive information.

20.6 Backup and Recovery

A production backup strategy should consider both the database and uploaded files.

Backing up the database alone may not preserve the actual ATP report files. Both data and media storage need a recovery plan.

21. KNOWN DEVELOPMENT ISSUES AND LESSONS

The following issues were reported during earlier development. They are included to help future maintainers recognize recurring problem patterns; they should not be interpreted as confirmation that every issue remains present.

21.1 Permission Mismatches

Some write operations were restricted to Inspector-only permissions, which prevented Admin or Team Lead users from performing operations they expected to access.

Lesson: Check backend permission classes and endpoint configuration when an authenticated user receives a `403 Forbidden` response.

21.2 Empty Fields in Search Details

ATP and Chloragel detail popups previously displayed blank fields or em dashes even when the underlying records contained information.

Lesson: Inspect the API response and compare its field names and object structure with the frontend's property mappings.

21.3 Incorrect Daily Record Dates

Records created around midnight previously appeared under an unexpected day in the combined records view.

Lesson: Investigate timezone handling, date conversion, and query boundaries before making changes to the displayed date.

21.4 Numeric IDs Instead of Names

Some views displayed identifiers instead of human-readable names for related users or inspection templates.

Lesson: Resolve the relevant relationship in the response or template and display the appropriate name.

21.5 Fresh Deployment Databases

Creating a new PostgreSQL database does not copy the existing development records.

Lesson: Treat schema migrations, account creation, and data migration as separate tasks.

21.6 Static Files and Media Files

Static assets and uploaded documents have different storage requirements.

Lesson: Configure persistent media storage for uploads and the appropriate static-file mechanism for application assets.

22. TROUBLESHOOTING GUIDE

Problem	Possible cause	Recommended investigation
Login fails	Invalid credentials or authentication configuration	Inspect the login response and authentication settings
API returns 401	Missing or invalid authentication token	Check token generation, expiration, and request headers
API returns 403	Permission denied	Inspect the endpoint's permission classes and user groups
API returns 400	Invalid request data	Inspect the response body and serializer validation
API returns 500	Unhandled backend exception	Read the Django server logs and reproduce the request
CSS does not load	Incorrect static configuration or path	Inspect browser Network tools and static settings
Records are missing	Filters, date handling, or database selection	Verify the request parameters and database contents
Search details are blank	Incorrect frontend field mapping	Compare the API response with the detail renderer

PDF generation fails	Report data or PDF-generation error	Inspect server logs and reporting code
Uploaded files disappear	Non-persistent media storage	Check the configured storage backend
Management command fails	Command missing or environment misconfigured	Verify the command implementation and required variables
Local login differs from deployment	Different database or account setup	Check which database each environment uses

22.1 Recommended Debugging Sequence

For a page or API defect:

1. Reproduce the issue.
2. Open the browser Developer Tools.
3. Inspect the Console for JavaScript errors.
4. Inspect the Network tab for failed requests.
5. Review the request payload and API response.
6. Inspect the Django server logs.
7. Review the relevant view, serializer, and permission class.
8. Check model relationships and database contents.
9. Review the template or JavaScript mapping.
10. Retest the affected workflow and related functionality.

This sequence helps distinguish frontend presentation defects from API, permission, business logic, and database problems.

23. OPERATIONAL PROCEDURES

23.1 Running the Application Locally

Activate the project's virtual environment and install its dependencies.

python -m pip install -r requirements.txt

Check the Django configuration:

python manage.py check

Apply database migrations:

python manage.py migrate

Start the development server:

python manage.py runserver

The default local address is:

`http://127.0.0.1:8000/`

The login page and dashboard routes depend on the configured URL patterns.

23.2 Before Committing Changes

Before pushing a change to GitHub:

- Run relevant tests.
- Run `python manage.py check`.
- Inspect modified files.
- Review migrations if models were changed.
- Confirm no secrets or private data are being committed.
- Test the relevant user roles.
- Verify that the frontend still communicates with the expected API.
- Confirm that error states are handled appropriately.

23.3 Before Deploying

Before deploying a new version:

- Verify environment variables.
- Confirm database connectivity.
- Apply migrations in a controlled manner.
- Check static-file configuration.
- Verify uploaded-file persistence.
- Test login and protected API endpoints.

- Test PDF generation.
- Review application logs.

24. FUTURE ENHANCEMENTS

The following items are possible future improvements, not claims that these features are already implemented.

24.1 Advanced Dashboard Analytics

Potential enhancements include:

- Daily and monthly sanitation summaries.
- Plant-level comparison charts.
- Titration trend analysis.
- Inspection completion statistics.
- Corrective action summaries.

24.2 Notifications

Possible features include:

- Submission reminders.
- Notifications for failed results.
- Overdue corrective action alerts.
- Email notifications for important events.

24.3 Reporting Improvements

Potential improvements include:

- More customizable PDF reports.
- Additional date-range filters.
- Plant-specific report templates.
- Summary statistics and charts.
- Improved export formats.

24.4 Automated Document Processing

Future versions could investigate extracting information from uploaded documents or images.

Any OCR feature should be designed with appropriate validation because extracted values may be inaccurate and should not silently replace verified records.

24.5 Security Improvements

Potential improvements include:

- Expanded automated permission tests.
- More detailed audit coverage.
- Better file-access controls.
- Formal backup and recovery testing.
- Security monitoring.
- Additional input validation.

24.6 Maintainability Improvements

Potential improvements include:

- Expanded API documentation.
- More automated integration tests.
- Centralized error handling.
- Clearer separation of business services.
- Automated deployment checks.
- Source-verified database documentation.

25. GLOSSARY

Term	Definition
API	Application Programming Interface; a defined way for clients to communicate with backend functionality
Backend	Server-side code that processes requests and applies business rules
Frontend	User-facing interface displayed in the browser

Django ORM	Django's object-relational mapper for database operations
DRF	Django REST Framework, used to build APIs
JWT	JSON Web Token, a token format used for authentication in configured systems
Model	A Django class representing a data entity
Primary key	A unique identifier for a database record
Foreign key	A field linking one record to another
Serializer	A DRF component for validating input and representing data
Migration	A versioned database schema change
Middleware	A layer that processes requests and responses
Authentication	Verification of a user's identity
Authorization	Determination of what an authenticated user may do
Static files	Application assets such as CSS and JavaScript

Media files	Files uploaded by users
CRUD	Create, Read, Update, Delete
PDF	Portable Document Format
SQLite	A file-based relational database
PostgreSQL	A relational database management system
WSGI	Web Server Gateway Interface used to connect Python web applications to compatible servers
Gunicorn	A WSGI server commonly used to serve Django applications
Audit log	A record of supported application events and actions
TCA report	In this document, a comprehensive technical architecture report

26. SOURCE-CODE VERIFICATION CHECKLIST

This section is especially important if the document will be used for a formal handover, technical review, or project assessment.

The report explains the documented architecture, but a fully verified technical specification must be checked against the current source code.

26.1 Application Configuration

Review:

- `config/settings.py`
- `config/urls.py`
- `config/wsgi.py`
- `requirements.txt`

Verify:

- Installed applications.
- Database configuration.
- Authentication settings.
- REST Framework settings.
- Static and media configuration.
- Allowed hosts and security settings.
- Exact dependency versions.

26.2 Database Schema

Inspect each application's:

- `models.py`
- `migrations/` directory.

Document:

- Exact model names.
- Every field and its type.
- Primary keys.
- Foreign keys.
- Nullability.
- Default values.
- Unique constraints.
- Deletion behavior.
- Database indexes.
- Model relationships.

Use these definitions to create the final physical entity-relationship diagram.

26.3 API Routes and Permissions

Inspect:

- `config/urls.py`
- Application-specific `urls.py`
- `views.py`
- `serializers.py`
- `accounts/permissions.py`

Verify:

- Exact page routes.
- Exact API routes.
- Supported HTTP methods.
- Request fields.
- Response fields.
- Authentication requirements.
- Permission requirements.
- Validation rules.
- Error responses.

26.4 Frontend and Navigation

Inspect:

- `templates/base.html`
- `templates/dashboard.html`
- All module templates.
- JavaScript embedded in templates.
- Static JavaScript and CSS files.

Verify:

- Actual navigation links.
- Buttons and form behavior.
- API endpoints called by each page.
- Authentication-token handling.
- Detail-popup field mappings.
- Loading and error states.

26.5 Reporting and File Storage

Inspect the ATP report implementation and PDF-generation helpers.

Verify:

- Allowed file types.
- File-size limits.
- Upload permissions.
- Media storage configuration.
- PDF response behavior.
- Date and plant filtering.
- Handling of missing values.
- User access to uploaded files.

26.6 Testing and Deployment

Inspect:

- Test modules.
- Environment configuration.
- Deployment configuration.
- Any custom management commands.

Verify that the documented setup commands actually exist and behave as described.

Formal Documentation Status

The current document is a project-context-based technical blueprint. It should be labelled as a source-verified architecture specification only after the route inventory, entity relationships, field definitions, and configuration have been compared with the current repository.

27. CONCLUSION

The Sanitation Record Management System is a modular Django web application designed to centralize sanitation activities and provide structured access to operational records.

The documented architecture combines a template-driven frontend, Django REST Framework APIs, backend business logic, relational data storage, file handling, PDF reporting, and role-based permissions.

Its principal functional areas are plant management, sanitizer and Chloragel titrations, daily inspections, ATP reports, combined daily records, search, corrective actions, user management, and audit logging.

The database design connects users and operational records through relationships, while plant and date context support unified record retrieval. Django's ORM provides the data-access layer, and APIs allow frontend components to communicate with backend functionality.

The most important technical maintenance principles are:

1. Keep frontend presentation separate from backend business rules.
2. Enforce authorization on the backend.
3. Validate data before saving it.
4. Preserve consistent date and timezone handling.
5. Resolve database relationships into meaningful display values.
6. Protect credentials, databases, and uploaded files.
7. Test the complete workflow after changes.
8. Keep technical documentation aligned with the current source code.

This blueprint provides a foundation for project onboarding, development handover, troubleshooting, future enhancements, and technical review.

DOCUMENT END

Sanitation Record Management System Technical Blueprint & Comprehensive Architecture Report Prepared by Bandish Vaidya